

ЗАДАНИЕ

Построение и тестирование конечноэлементной вычислительной схемы с использованием векторных базисных функций на параллелепипедах и прямоугольниках.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. МАТЕМАТИЧЕСКАЯ ПОСТАНОВКА

Решение задачи в трехмерных декартовых координатах, удовлетворяющей расчетной области Ω , будем искать из следующего уравнения:

$$\operatorname{rot} \frac{1}{\mu} \operatorname{rot} \vec{\mathbf{A}} + \vec{\mathbf{A}} = \vec{\mathbf{J}} \quad (1.1)$$

Эквивалентная вариационная постановка для уравнения (1.1) имеет вид:

$$\int_{\Omega} \frac{1}{\mu} \operatorname{rot} \vec{\mathbf{A}} \operatorname{rot} \vec{\Psi} d\Omega + \int_{\Omega} \vec{\mathbf{A}} \vec{\Psi} d\Omega = \int_{\Omega} \vec{\mathbf{J}} \vec{\Psi} d\Omega. \quad (1.2)$$

1.2. ПРИНЦИПЫ ПОСТРОЕНИЯ ЛОКАЛЬНЫХ ВЕКТОРОВ, МАТРИЦ ЖЕСТКОСТИ И МАСС В ДВУМЕРНОМ ВЕКТОРНОМ МКЭ

Решение будем искать, используя билинейные базисные вектор-функции ??, задаваемые линейными функциями на прямоугольниках $\Omega_{rs} = [x_r, x_{r+1}] \times [y_s, y_{s+1}]$ следующего вида:

$$\begin{aligned} \vec{\psi}_1 &= \begin{pmatrix} 0 \\ \frac{x_{r+1}-x}{h_x} \end{pmatrix}, & \vec{\psi}_2 &= \begin{pmatrix} 0 \\ \frac{x-x_r}{h_x} \end{pmatrix}, \\ \vec{\psi}_3 &= \begin{pmatrix} \frac{y_{s+1}-y}{h_y} \\ 0 \end{pmatrix}, & \vec{\psi}_4 &= \begin{pmatrix} \frac{y-y_s}{h_y} \\ 0 \end{pmatrix}. \end{aligned}$$

где $h_x = x_{r+1} - x_r$, $h_y = y_{s+1} - y_s$.

Локальная матрица жёсткости $\hat{\mathbf{G}}$ на векторном прямоугольнике при $\bar{\mu} = \text{const}$ принимает вид:

$$\hat{\mathbf{G}} = \frac{1}{\bar{\mu}} \begin{pmatrix} \frac{h_y}{h_x} & -\frac{h_y}{h_x} & -1 & 1 \\ -\frac{h_y}{h_x} & \frac{h_y}{h_x} & 1 & -1 \\ -1 & 1 & \frac{h_x}{h_y} & -\frac{h_x}{h_y} \\ 1 & -1 & -\frac{h_x}{h_y} & \frac{h_x}{h_y} \end{pmatrix}.$$

Локальная матрица масс $\hat{\mathbf{M}}$ на векторном прямоугольнике при $\bar{\gamma} = \text{const}$ принимает вид:

$$\hat{\mathbf{M}} = \bar{\gamma} \frac{h_x h_y}{6} \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 2 & 0 & 0 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{pmatrix}.$$

Локальный вектор правой части $\hat{\mathbf{b}}$ на векторном прямоугольнике принимает вид:

$$\hat{\mathbf{b}} = \hat{\mathbf{M}} \begin{pmatrix} F_y(x_r, y_{s+\frac{1}{2}}) \\ F_y(x_{r+1}, y_{s+\frac{1}{2}}) \\ F_x(x_{r+\frac{1}{2}}, y_s) \\ F_x(x_{r+\frac{1}{2}}, y_{s+1}) \end{pmatrix}.$$

1.3. ПРИНЦИПЫ ПОСТРОЕНИЯ ЛОКАЛЬНЫХ ВЕКТОРОВ, МАТРИЦ ЖЕСТКОСТИ И МАСС В ТРЕХМЕРНОМ ВЕКТОРНОМ МКЭ

Решение будем искать, используя билинейные базисные вектор-функции, задаваемые линейными функциями на параллелепипеде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ следующего вида:

$$X_1(x) = \frac{x_{r+1} - x}{x_{r+1} - x_r}, \quad X_2(x) = \frac{x - x_r}{x_{r+1} - x_r}, \quad (1.3)$$

$$Y_1(y) = \frac{y_{s+1} - y}{y_{s+1} - y_s}, \quad Y_2(y) = \frac{y - y_s}{y_{s+1} - y_s}, \quad (1.4)$$

$$Z_1(z) = \frac{z_{p+1} - z}{z_{p+1} - z_p}, \quad Z_2(z) = \frac{z - z_p}{z_{p+1} - z_p}. \quad (1.5)$$

Тогда базисная вектор-функция, ассоциированная с ребром $\{y = y_s, z = z_p\}$ имеет вид:

$$\vec{\psi}_1 = \begin{pmatrix} \frac{y_{s+1} - y}{h_y} \cdot \frac{z_{p+1} - z}{h_z} \\ 0 \\ 0 \end{pmatrix},$$

где $h_y = y_{s+1} - y_s$ $h_z = z_{p+1} - z_p$.

Полный список базисных вектор-функций на параллелепипеде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ можно записать в виде:

$$\vec{\psi}_1 = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{\psi}_2 = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{\psi}_3 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\vec{\psi}_4 = \begin{pmatrix} Y_2 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{\psi}_5 = \begin{pmatrix} 0 \\ X_1 \cdot Z_1 \\ 0 \end{pmatrix}, \quad \vec{\psi}_6 = \begin{pmatrix} 0 \\ X_2 \cdot Z_1 \\ 0 \end{pmatrix},$$

$$\vec{\psi}_7 = \begin{pmatrix} 0 \\ X_1 \cdot Z_2 \\ 0 \end{pmatrix}, \quad \vec{\psi}_8 = \begin{pmatrix} 0 \\ X_2 \cdot Z_2 \\ 0 \end{pmatrix}, \quad \vec{\psi}_9 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\vec{\psi}_{10} = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{\psi}_{11} = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{\psi}_{12} = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

Формулы для вычисления глобальных матриц жёсткости $\mathbf{G}$ и масс $\mathbf{M}$, определяющих глобальную матрицу $\mathbf{C} = \mathbf{G} + \mathbf{M}$ конечноэлементной СЛАУ имеют вид:

$$G_{ij} = \int_{\Omega} \frac{1}{\mu} \text{rot} \vec{\Psi}_i \cdot \text{rot} \vec{\Psi}_j d\Omega, \quad M_{ij} = \int_{\Omega} \gamma \vec{\Psi}_i \cdot \vec{\Psi}_j d\Omega. \quad (1.6)$$

Компоненты глобального вектора $\mathbf{b}$ конечноэлементной СЛАУ определяются соотношением:

$$b_i = \int_{\Omega} \vec{\mathbf{F}} \cdot \vec{\Psi}_i d\Omega. \quad (1.7)$$

Локальная матрица жёсткости $\hat{\mathbf{G}}$ на векторном параллелепипеде при $\bar{\mu} = \text{const}$ принимает вид:

$$\hat{\mathbf{G}} = \frac{1}{\bar{\mu}} \begin{pmatrix} \frac{h_x h_y}{6h_z} \mathbf{G}_1 + \frac{h_x h_z}{6h_y} \mathbf{G}_2 & -\frac{h_z}{6} \mathbf{G}_2 & \frac{h_y}{6} \mathbf{G}_3 \\ -\frac{h_z}{6} \mathbf{G}_2 & \frac{h_x h_y}{6h_z} \mathbf{G}_1 + \frac{h_y h_z}{6h_x} \mathbf{G}_2 & -\frac{h_x}{6} \mathbf{G}_1 \\ \frac{h_y}{6} \mathbf{G}_3^T & -\frac{h_x}{6} \mathbf{G}_1 & \frac{h_x h_z}{6h_y} \mathbf{G}_1 + \frac{h_y h_z}{6h_x} \mathbf{G}_2 \end{pmatrix},$$

где

$$\mathbf{G}_1 = \begin{pmatrix} 2 & 1 & -2 & -1 \\ 1 & 2 & -1 & -2 \\ -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \end{pmatrix}, \mathbf{G}_2 = \begin{pmatrix} 2 & -2 & 1 & -1 \\ -2 & 2 & -1 & 2 \\ 1 & -1 & 2 & -2 \\ -1 & 1 & -2 & 2 \end{pmatrix},$$

$$\mathbf{G}_3 = \begin{pmatrix} -2 & 2 & -1 & 1 \\ -1 & 1 & -2 & 2 \\ 2 & -2 & 1 & -1 \\ 1 & -1 & 2 & -2 \end{pmatrix}.$$

Матрицу $\hat{\mathbf{M}}$ можно представить в виде:

$$\mathbf{M} = \begin{pmatrix} \mathbf{D} & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{D} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{D} \end{pmatrix},$$

где $\mathbf{O}$ - матрица 4×4 , состоящая из нулей подматрица, а $\mathbf{D}$ определяется следующим образом:

$$\mathbf{D} = \begin{pmatrix} 4 & 2 & 2 & 1 \\ 2 & 4 & 1 & 2 \\ 2 & 1 & 4 & 2 \\ 1 & 2 & 2 & 4 \end{pmatrix}.$$

Локальный вектор правой части определяется в виде $\hat{\mathbf{b}}_i = \hat{\mathbf{M}} \cdot \hat{\mathbf{f}}_i$, где

$$\hat{\mathbf{f}}_i = \overrightarrow{\mathbf{F}}(\hat{x}_c, \hat{y}_c, \hat{z}_c) \cdot \overrightarrow{\mathbf{v}}/l, \quad (1.8)$$

$\hat{x}_c, \hat{y}_c, \hat{z}_c$ - координаты центра ребра, $\overrightarrow{\mathbf{v}}$ - вектор, направленный вдоль ребра Γ и сонаправленный с базисной вектор-функцией $\overrightarrow{\psi}_i$, l - длина данного вектора.

2. ИССЛЕДОВАНИЯ 3D

2.1. ПРЕДВАРИТЕЛЬНОЕ ОПИСАНИЕ

Проведем сначала тестирование разработанной программы по векторному МКЭ на работоспособность. Образец расчетной области изображен на рисунке 3.1. Это область $\Omega = [0.0, 2.0]_x \times [0.0, 2.0]_y \times [0.0, 2.0]_z$, она содержит 54 ребра, на всех границах будем задавать первые краевые условия. Середины наших элементов обозначены красным цветом, в этих точках мы будем рассматривать точность наших решений для оценки точности программы.

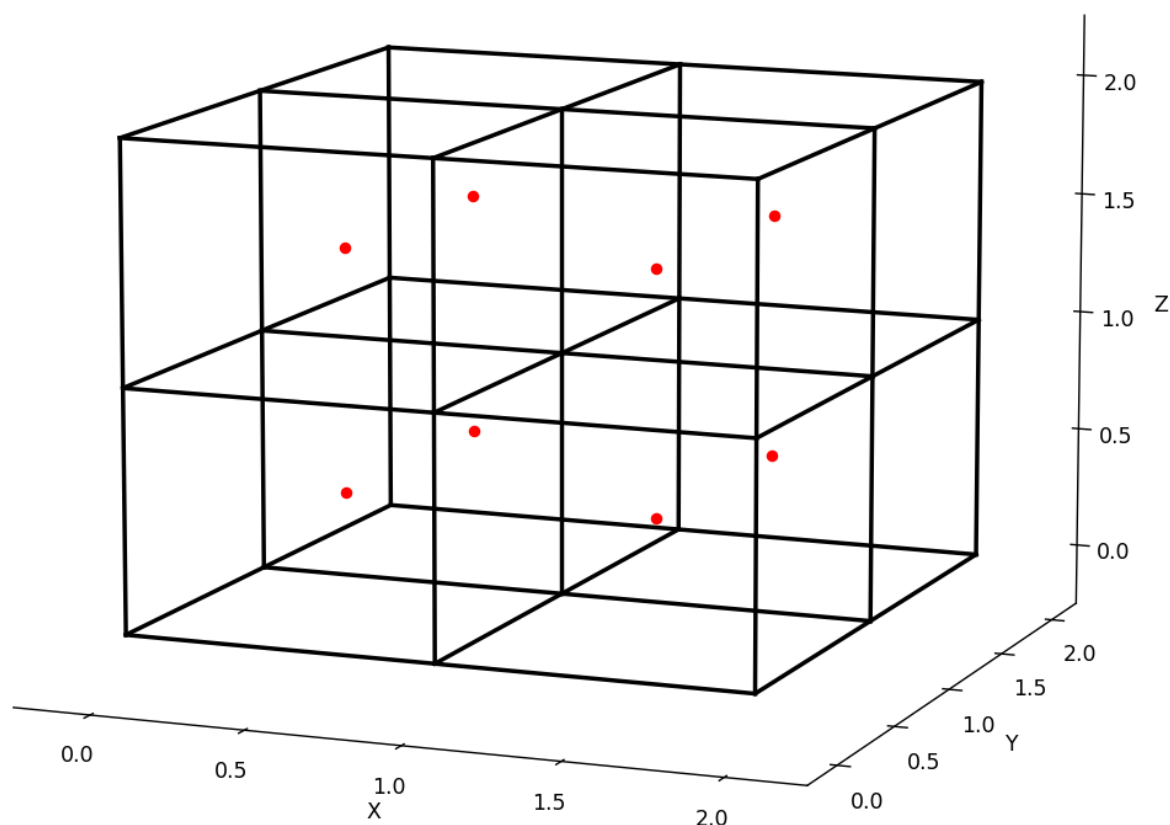


Рисунок 2.1 – Расчетная область

2.2. ТЕСТИРОВАНИЕ НА ПОРЯДОК АППРОКСИМАЦИИ

Тестирование будем проводить на дифференциальном уравнении (3.1) :

$$\operatorname{rot} \left(\frac{1}{\mu} \operatorname{rot} \vec{\mathbf{A}} \right) + \gamma \vec{\mathbf{A}} = \vec{\mathbf{F}} \quad (2.1)$$

Таблица 2.1 – Тестирование при $\vec{A} = (1.0, 1.0, 1.0)^T$, $\vec{F} = (1.0, 1.0, 1.0)^T$,
 $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.2 – Тестирование при $\vec{A} = (y, z, x)^T$, $\vec{F} = (y, z, x)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.3 – Тестирование при $\vec{A} = (1 + y + x; 1 + x + z; 1 + x + y)^T$,
 $\vec{F} = (1 + y + x; 1 + x + z; 1 + x + y)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.00000000E+000; 2.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	3.00000000E+000; 2.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	4.00000000E+000; 3.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	2.00000000E+000; 3.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 4.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	4.00000000E+000; 4.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.4 – Тестирование при $\vec{A} = (y - z; x - z; x - y)^T$,
 $\vec{F} = (y - z; x - z; x - y)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	0.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 0.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	-1.00000000E+000; -1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	-1.00000000E+000; 0.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	0.00000000E+000; -1.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.5 – Тестирование при $\vec{A} = (y \cdot z; x \cdot z; x \cdot y)^T$,
 $\vec{F} = (y \cdot z; x \cdot z; x \cdot y)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	2.50000000E-001; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	7.50000000E-001; 2.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	7.50000000E-001; 7.50000000E-001; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	7.50000000E-001; 7.50000000E-001; 2.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	7.50000000E-001; 2.25000000E+000; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	2.25000000E+000; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	2.25000000E+000; 2.25000000E+000; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 2.6 – Тестирование при $\vec{A} = (y^2; z^2; x^2)^T$,
 $\vec{F} = (y^2 - 2; z^2 - 2; x^2 - 2)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.11111111E-001
(0.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.00000000E+000; 1.00000000E+000
(1.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.00000000E+000; 1.11111111E-001
(0.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.11111111E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.11111111E-001; 1.11111111E-001
(0.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.11111111E-001; 1.00000000E+000
(1.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.11111111E-001; 1.11111111E-001

Таблица 2.7 – Тестирование при $\vec{A} = (y^2 + z^2; x^2 + z^2; x^2 + y^2)^T$,
 $\vec{F} = (y^2 + z^2 - 4; x^2 + z^2 - 4; x^2 + y^2 - 4)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 2.00000000E-001; 2.00000000E-001
(0.5; 1.5; 0.5)	3.00000000E+000; 1.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.00000000E+000; 2.00000000E-001
(1.5; 1.5; 0.5)	3.00000000E+000; 3.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.11111111E-001
(0.5; 0.5; 1.5)	3.00000000E+000; 3.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 5.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.11111111E-001; 2.00000000E-001
(0.5; 1.5; 1.5)	5.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.11111111E-001; 2.00000000E-001; 2.00000000E-001
(1.5; 1.5; 1.5)	5.00000000E+000; 5.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.11111111E-001; 1.11111111E-001; 1.11111111E-001

Порядок аппроксимации равен 1.

2.3. ТЕСТИРОВАНИЕ НА ПОРЯДОК СХОДИМОСТИ

Таблица 2.8 – Тестирование при $\vec{A} = (y^2 z^2; x^2 z^2; x^2 y^2)^T$,

$$\vec{F} = (y^2 z^2 - 2 \cdot (y^2 + z^2); x^2 z^2 - 2 \cdot (x^2 + z^2); x^2 y^2 - 2 \cdot (x^2 + y^2))^T,$$

$$\mu = 1, \gamma = 1$$

Точка	Относительная погрешность	Порядок $\log_2 \left(\frac{\Delta_{i-1}}{\Delta_i} \right)$
$(\frac{4}{3}; \frac{4}{3}; \frac{4}{3})$	2.6562500E-001	—
	2.6562500E-001	
	2.6562500E-001	
	6.3476562E-002	2.065
	6.3476562E-002	
	6.3476562E-002	
	1.5686035E-002	2.017
	1.5686035E-002	
	1.5686035E-002	
	3.9100647E-003	2.004
	3.9100647E-003	
	3.9100647E-003	

Можем увидеть, что порядок сходимости стремится к 2.

3. ИССЛЕДОВАНИЯ 2D

3.1. ПРЕДВАРИТЕЛЬНОЕ ОПИСАНИЕ

Проведем сначала тестирование разработанной программы по векторному МКЭ на работоспособность. Образец расчетной области изображен на рисунке 3.1. Это область $\Omega = [0.0, 2.0]_x \times [0.0, 2.0]_y$, она содержит 12 ребер, на всех границах будем задавать первые краевые условия. Середины наших элементов обозначены красным цветом, в этих точках мы будем рассматривать точность наших решений для оценки точности программы.

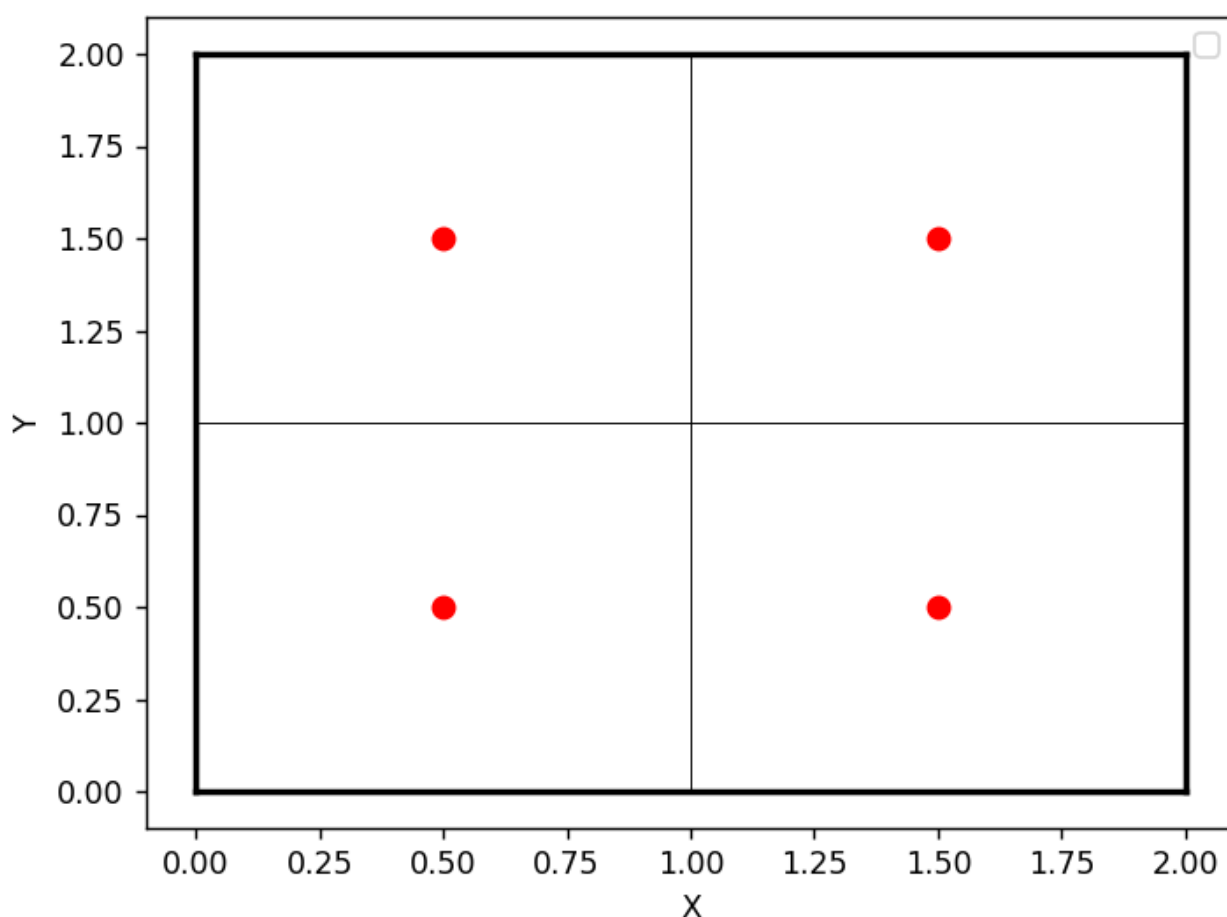


Рисунок 3.1 – Расчетная область

3.2. ТЕСТИРОВАНИЕ НА ПОРЯДОК АППРОКСИМАЦИИ

Тестирование будем проводить на дифференциальном уравнении (3.1) :

$$\text{rot} \left(\frac{1}{\mu} \text{rot} \vec{A} \right) + \gamma \vec{A} = \vec{F} \quad (3.1)$$

Таблица 3.1 – Тестирование при $\vec{A} = (1.0, 1.0)^T$, $\vec{F} = (1.0, 1.0)^T$, $\mu = 1$,
 $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(1.5; 0.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(0.5; 1.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(1.5; 1.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016

Таблица 3.2 – Тестирование при $\vec{A} = (y, x)^T$, $\vec{F} = (y, x)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(1.5; 0.5)	1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(0.5; 1.5)	5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(1.5; 1.5)	1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000

Таблица 3.3 – Тестирование при $\vec{A} = (1 + y + x; 1 + x)^T$,
 $\vec{F} = (1 + y + x; 1 + x)^T$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	2.00000000E+000; 1.50000000E+000	4.44089210E-016; 0.00000000E+000	2.22044605E-016; 0.00000000E+000
(1.5; 0.5)	3.00000000E+000; 2.50000000E+000	4.44089210E-016; 4.44089210E-016	1.48029737E-016; 1.77635684E-016
(0.5; 1.5)	3.00000000E+000; 1.50000000E+000	8.88178420E-016; 0.00000000E+000	2.96059473E-016; 0.00000000E+000
(1.5; 1.5)	4.00000000E+000; 2.50000000E+000	8.88178420E-016; 4.44089210E-016	2.22044605E-016; 1.77635684E-016

Таблица 3.4 – Тестирование при $\vec{A} = (y^2; x^2)^T$, $\vec{F} = (y^2 - 2; x^2 - 2)^T$, $\mu = 1$,
 $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000
(1.5; 0.5)	5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.11111111E-001
(0.5; 1.5)	2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.00000000E+000
(1.5; 1.5)	2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.11111111E-001

Порядок аппроксимации равен 1.

3.3. ТЕСТИРОВАНИЕ НА ПОРЯДОК СХОДИМОСТИ

Таблица 3.5 – Тестирование при $\vec{A} = (y^2; x^2)^T$, $\vec{F} = (y^2 - 2; x^2 - 2)^T$, $\mu = 1$,
 $\gamma = 1$

Точка	Относительная погрешность	Порядок $\log_2 \left(\frac{\Delta_{i-1}}{\Delta_i} \right)$
$(\frac{4}{3}; \frac{4}{3}; \frac{4}{3})$	1.2500000E-001	—
	1.2500000E-001	
	3.1250000E-002	2.000
	3.1250000E-002	
	7.8125000E-003	2.000
	7.8125000E-003	
	1.9531250E-003	2.000
	1.9531250E-003	

Можем увидеть, что порядок сходимости равен 2.

СПИСОК ЛИТЕРАТУРЫ

1. Ю.Г. Соловейчик, М.Э. Рояк, М.Г. Персова Метод конечных элементов для скалярных и векторных задач Учеб. пособие. — Новосибирск: Изд-во НГТУ, 2007 — 896 с.
2. А.Н. Тихонов, А.А. Самарский Уравнения математической физики: Учеб.пособие. / А.Н. Тихонов, А.А. Самарский — 6-е изд., — М: Изд-во МГУ, 1999 — 799 с.

ПРИЛОЖЕНИЕ. ТЕКСТ ПРОГРАММЫ.

Program.cs

```
1  #define TESTING
2  // RELEASE
3  // TESTING
4  #define NODRAWING
5  #define SOLVE2DIM
6  using Project;
7  using System.Globalization;
8  using Solver;
9  using Manager;
10 using System.Numerics;
11 using Processor;
12 using Solution;
13 using MathObjects;
14 using Functions;
15 using Grid;
16 using DataStructs;
17
18
19 CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;
20
21 string BordersInfo = Path.GetFullPath("../.../Data/Input/Borders.dat");
22 string AnswerPath = Path.GetFullPath("../.../Data/Output/");
23 string TimePath = Path.GetFullPath("../.../Data/Input/Time.dat");
24
25 FEM3D myFEM3D_test = new();
26 myFEM3D_test.ConstructMesh(3, 3, 3);
27 myFEM3D_test.GenerateArrays();
28 myFEM3D_test.ConstructMatrixAndVector();
29 myFEM3D_test.SetSolver(new LOS());
30 myFEM3D_test.Solve();
31 myFEM3D_test.TestOutput(AnswerPath);
32 myFEM3D_test.WriteData(AnswerPath);
33
34 return 0;
```

LocalMatrix3D.cs

```
1 namespace MathObjects;
2
3 public class LocalMatrixG3D (double mu, double hx, double hy, double hz) : Matrix
4 {
5     private readonly double _mu = mu;
6     private readonly double _hx = hx;
7     private readonly double _hy = hy;
8     private readonly double _hz = hz;
9
10    private readonly double[,] G1 = {{ 2.0D, 1.0D, -2.0D, -1.0D},
11                                     { 1.0D, 2.0D, -1.0D, -2.0D},
12                                     {-2.0D, -1.0D, 2.0D, 1.0D},
13                                     {-1.0D, -2.0D, 1.0D, 2.0D}};
14
15    private readonly double[,] G2 = {{ 2.0D, -2.0D, 1.0D, -1.0D},
16                                     {-2.0D, 2.0D, -1.0D, 1.0D},
17                                     { 1.0D, -1.0D, 2.0D, -2.0D},
18                                     {-1.0D, 1.0D, -2.0D, 2.0D}};
19
20    private readonly double[,] G3 = {{-2.0D, 2.0D, -1.0D, 1.0D},
21                                     {-1.0D, 1.0D, -2.0D, 2.0D},
22                                     { 2.0D, -2.0D, 1.0D, -1.0D},
23                                     { 1.0D, -1.0D, 2.0D, -2.0D}};
24
25    public override double this[int i, int j]
26    {
27        get
28        {
29            return (i / 4, j / 4)
30                switch
31                {
32                    (0, 0) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hx * _hz /
33                               ↪ (6.0D * _hy) * G2[i % 4, j % 4]) / _mu,
34                    (0, 1) or (1, 0) => -1.0D * (_hz / 6.0D) * G2[i % 4, j % 4] / _mu,
35                    (0, 2) => _hy / 6.0D * G3[i % 4, j % 4] / _mu,
36                    (1, 1) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hy * _hz /
37                               ↪ (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
38                    (1, 2) or (2, 1) => -1.0D * _hx / 6.0D * G1[i % 4, j % 4] / _mu,
```



```

37         (2, 0) => _hy / 6.0D * G3[j % 4, i % 4] / _mu,
38         (2, 2) => (_hx * _hz / (6.0D * _hy) * G1[i % 4, j % 4] + _hy * _hz /
        ↪ (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
39         _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
        ↪ range"),
40     };
41 }
42 set{}
43 }
44 }
45
46 public class LocalMatrixM3D(double gamma, double hx, double hy, double hz) : Matrix
47 {
48     private readonly double _gamma = gamma;
49     private readonly double _hx = hx;
50     private readonly double _hy = hy;
51     private readonly double _hz = hz;
52
53     private readonly double[,] D = {{4.0D, 2.0D, 2.0D, 1.0D},
54                                     {2.0D, 4.0D, 1.0D, 2.0D},
55                                     {2.0D, 1.0D, 4.0D, 2.0D},
56                                     {1.0D, 2.0D, 2.0D, 4.0D}};
57
58     public override double this[int i, int j]
59     {
60         get
61         {
62             return (i / 4, j / 4)
63                 switch
64                 {
65                     (0, 0) or (1, 1) or (2, 2) => _gamma * _hx * _hy * _hz / 36.0D * D[i % 4,
        ↪ j % 4],
66                     (0, 1) or (0, 2) or (1, 0) or (1, 2) or (2, 0) or (2, 1) => 0.0D,
67                     _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
        ↪ range"),
68                 };
69         }
70         set{}
71     }
72 }

```

LocalVector3D.cs

```
1 using static Functions.Function;
2
3 namespace MathObjects;
4
5
6 public class LocalVector3D : Vector
7 {
8     private readonly double x0;
9     private readonly double x1;
10    private readonly double y0;
11    private readonly double y1;
12    private readonly double z0;
13    private readonly double z1;
14    private readonly double xm;
15    private readonly double ym;
16    private readonly double zm;
17    private readonly double t;
18
19    private LocalMatrixM3D M;
20
21    public override double this[int i]
22    {
23        get
24        {
25            double ScalarMult((double, double, double) a, (double, double, double) b)
26            => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;
27
28            (double, double, double) Divide ((double, double, double) v, double sc)
29            => (v.Item1 / sc, v.Item2 / sc, v.Item3 / sc);
30
31            double Length((double, double, double) v)
32            => Math.Sqrt(Math.Pow(v.Item1, 2) + Math.Pow(v.Item2, 2) + Math.Pow(v.Item3,
33                ↵ 2));
34
35            // Очень спорно.
36            List<double> vect = [
37                ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
38                ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
```

```

38         ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
39         ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),
40
41         ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
42         ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
43         ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
44         ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),
45
46         ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
47         ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
48         ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
49         ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
50     ];
51
52     double ans = 0.0D;
53     for (int j = 0; j < vect.Count; j++)
54         ans += M[i, j] * vect[j];
55     return ans;
56 }
57 }
58
59 public LocalVector3D(double x0, double x1, double y0, double y1, double z0, double
↵ z1, double t)
60 {
61     this.x0 = x0;
62     this.x1 = x1;
63     this.y0 = y0;
64     this.y1 = y1;
65     this.z0 = z0;
66     this.z1 = z1;
67     this.t = t;
68     xm = 0.5D * (x1 + x0);
69     ym = 0.5D * (y1 + y0);
70     zm = 0.5D * (z1 + z0);
71     M = new(1.0, x1 - x0, y1 - y0, z1 - z0);
72     Generate();
73 }
74
75 private void Generate()
76 {

```

```

77     double ScalarMult((double, double, double) a, (double, double, double) b)
78     => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;
79
80     List<double> vect = [
81         ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
82         ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
83         ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
84         ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),
85
86         ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
87         ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
88         ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
89         ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),
90
91         ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
92         ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
93         ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
94         ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
95     ];
96
97     double ans = 0.0D;
98     for (int i = 0; i < vect.Count; i++)
99     {
100         for (int j = 0; j < vect.Count; j++)
101             ans += M[i, j] * vect[j];
102         ans = 0.0D;
103     }
104 }
105 }

```

FEM3D.cs

```

1 namespace Project;
2
3 using System.Collections.Immutable;
4 using System.Numerics;
5 using MathObjects;
6 using Solver;
7 using Grid;
8 using DataStructs;

```

```

9  using System.Diagnostics;
10 using Functions;
11 using System.ComponentModel.DataAnnotations;
12 using System.Timers;
13
14 public class FEM3D : FEM
15 {
16     public ArrayOfRibs ribsArr;
17
18     // Maybe private?
19     public List<Layer> Layers;
20
21     public FEM3D()
22     {
23         Layers = [];
24         mesh = new Mesh3Dim
25         {
26             nodesX = [],
27             nodesY = [],
28             nodesZ = []
29         };
30     }
31
32     public void ConstructMesh(int nx, int ny, int nz)
33     {
34         if (mesh == null) throw new ArgumentNullException("Mesh is null");
35
36         double hx = 2.0D;
37         double hy = 2.0D;
38         double hz = 2.0D;
39
40         mesh.nodesX = [];
41         mesh.nodesY = [];
42         mesh.nodesZ = [];
43
44         for (int i = 0; i < nx; i++)
45         {
46             double stepx = hx / (nx - 1);
47             mesh.nodesX.Add(i * stepx);
48         }

```

```

49
50     for (int i = 0; i < ny; i++)
51     {
52         double stepy = hy / (ny - 1);
53         mesh.nodesY.Add(i * stepy);
54     }
55
56     for (int i = 0; i < nz; i++)
57     {
58         double stepz = hz / (nz - 1);
59         mesh.nodesZ.Add(i * stepz);
60     }
61
62     timeMesh = [1.0D];
63 }
64
65
66
67 public void GenerateArrays()
68 {
69     if (mesh is null) throw new ArgumentNullException("mesh is null!");
70     pointsArr = MeshGenerator.GenerateListOfPoints(mesh);
71     ribsArr = MeshGenerator.GenerateListOfRibs(mesh, pointsArr);
72     elemsArr = MeshGenerator.GenerateListOfElems(mesh, ribsArr);
73     bordersArr = MeshGenerator.GenerateListOfBorders(mesh);
74 }
75
76 public void AddField(Layer layer)
77 {
78     ArgumentNullException.ThrowIfNull(layer);
79     Layers.Add(layer);
80 }
81
82 public void CommitFields()
83 {
84     if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
85
86     foreach (var layer in Layers)
87     {
88         for (int i = 0; i < elemsArr.Length; i++)

```

```

89         {
90             double minz = Math.Min(ribsArr[elemsArr[i][^1]].a.Z,
91                                     ↪ ribsArr[elemsArr[i][^1]].b.Z);
92             double maxz = Math.Max(ribsArr[elemsArr[i][^1]].a.Z,
93                                     ↪ ribsArr[elemsArr[i][^1]].b.Z);
94             if (layer.z0 <= minz && maxz <= layer.z1)
95                 elemsArr.sigmai[i] = layer.sigma;
96         }
97     }
98 }
99
100 public void ConstructMatrixAndVector()
101 {
102     if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
103     if (bordersArr is null) throw new ArgumentNullException("bordersArr is null");
104
105     var sparseMatrix = new GlobalMatrix(ribsArr.Count);
106     Generator.BuildPortait(ref sparseMatrix, ribsArr.Count, elemsArr);
107
108     var G = new GlobalMatrix(sparseMatrix);
109     Generator.FillMatrixG(ref G, ribsArr, elemsArr);
110
111     var M = new GlobalMatrix(sparseMatrix);
112     Generator.FillMatrixM(ref M, ribsArr, elemsArr);
113
114     Matrix = G + M;
115
116     var b = new GlobalVector(ribsArr.Count);
117     Generator.FillVector3D(ref b, ribsArr, elemsArr, 0.0);
118     Vector = b;
119
120     Generator.ConsiderBoundaryConditions(ref Matrix, ref Vector, ribsArr, bordersArr,
121                                         ↪ 0.0D);
122 }
123
124 public void Solve()
125 {
126     if (solver is null) throw new ArgumentNullException("Solver is null");
127     if (Matrix is null) throw new ArgumentNullException("Matrix is null");
128     if (Vector is null) throw new ArgumentNullException("Vector is null");

```

```

126     Solutions = new GlobalVector[timeMesh.Length];
127     Discrepancy = new GlobalVector[timeMesh.Length];
128     (Solutions[0], Discrepancy[0]) = solver.Solve(Matrix, Vector);
129 }
130
131 public void TestOutput(string path)
132 {
133     using var sw = new StreamWriter(path + "/Answer3D/Answer_Test.txt");
134
135     var absDiscX = 0.0D;
136     var absDiscY = 0.0D;
137     var absDiscZ = 0.0D;
138
139     var absDivX = 0.0D;
140     var absDivY = 0.0D;
141     var absDivZ = 0.0D;
142
143     var relDiscX = 0.0D;
144     var relDiscY = 0.0D;
145     var relDiscZ = 0.0D;
146
147     var relDivX = 0.0D;
148     var relDivY = 0.0D;
149     var relDivZ = 0.0D;
150
151     var squareDiffX = 0.0D;
152     var squareDiffY = 0.0D;
153     var squareDiffZ = 0.0D;
154
155     int iter = 0;
156
157     foreach (var elem in elemsArr)
158     {
159         int[] elem_local = [elem[0], elem[3], elem[8], elem[11],
160                             elem[1], elem[2], elem[9], elem[10],
161                             elem[4], elem[5], elem[6], elem[7]];
162
163         var x = 0.5D * (ribsArr[elem_local[0]].a.X + ribsArr[elem_local[0]].b.X);
164         var y = 0.5D * (ribsArr[elem_local[4]].a.Y + ribsArr[elem_local[4]].b.Y);
165         var z = 0.5D * (ribsArr[elem_local[8]].a.Z + ribsArr[elem_local[8]].b.Z);

```



```

166
167 sw.WriteLine($"Points {x:E15} {y:E15} {z:E15}");
168
169 var eps = (x - ribsArr[elem_local[0]].a.X) / (ribsArr[elem_local[0]].b.X -
↪ ribsArr[elem_local[0]].a.X);
170 var nu = (y - ribsArr[elem_local[4]].a.Y) / (ribsArr[elem_local[4]].b.Y -
↪ ribsArr[elem_local[4]].a.Y);
171 var khi = (z - ribsArr[elem_local[8]].a.Z) / (ribsArr[elem_local[8]].b.Z -
↪ ribsArr[elem_local[8]].a.Z);
172
173 double[] q = [Solutions[0][elem_local[0]], Solutions[0][elem_local[1]],
↪ Solutions[0][elem_local[2]], Solutions[0][elem_local[3]],
174 Solutions[0][elem_local[4]], Solutions[0][elem_local[5]],
↪ Solutions[0][elem_local[6]], Solutions[0][elem_local[7]],
175 Solutions[0][elem_local[8]], Solutions[0][elem_local[9]],
↪ Solutions[0][elem_local[10]], Solutions[0][elem_local[11]]];
176
177 var ans = BasisFunctions3DVec.GetValue(eps, nu, khi, q);
178 var theorValue = Function.A(x, y, z, 0.0D);
179
180 sw.WriteLine($"FEM A {ans.Item1:E15} {ans.Item2:E15} {ans.Item3:E15}");
181 sw.WriteLine($"Theor {theorValue.Item1:E15} {theorValue.Item2:E15}
↪ {theorValue.Item3:E15}");
182
183 var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
184 var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
185 var currAbsDiscZ = Math.Abs(ans.Item3 - theorValue.Item3);
186
187 squareDiffX += currAbsDiscX * currAbsDiscX;
188 squareDiffY += currAbsDiscY * currAbsDiscY;
189 squareDiffZ += currAbsDiscZ * currAbsDiscZ;
190
191 var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
192 var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
193 var currRelDiscZ = currAbsDiscZ / Math.Abs(theorValue.Item3);
194
195 sw.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}
↪ {currAbsDiscZ:E15}");
196 sw.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}
↪ {currRelDiscZ:E15}\n");

```

```

197
198         absDiscX += currAbsDiscX;
199         absDiscY += currAbsDiscY;
200         absDiscZ += currAbsDiscZ;
201
202         absDivX += theorValue.Item1;
203         absDivY += theorValue.Item2;
204         absDivZ += theorValue.Item3;
205
206         relDiscX += currRelDiscX * currRelDiscX;
207         relDiscY += currRelDiscY * currRelDiscY;
208         relDiscZ += currRelDiscZ * currRelDiscZ;
209
210         relDivX += theorValue.Item1 * theorValue.Item1;
211         relDivY += theorValue.Item2 * theorValue.Item2;
212         relDivZ += theorValue.Item3 * theorValue.Item3;
213
214         iter++;
215     }
216     sw.WriteLine($"Avg disc: {absDiscX / iter:E15} {absDiscY / iter:E15} {absDiscZ /
↪ iter:E15}");
217     sw.WriteLine($"Rel disc: {Math.Sqrt(relDiscX / relDivX):E15} {Math.Sqrt(relDiscY
↪ / relDivY):E15} {Math.Sqrt(relDiscZ / relDivZ):E15}");
218     sw.WriteLine($"SK0: {Math.Sqrt(squareDiffX / elemsArr.Length):E15}
↪ {Math.Sqrt(squareDiffY / elemsArr.Length):E15} {Math.Sqrt(squareDiffZ /
↪ elemsArr.Length):E15}");
219     sw.Close();
220 }
221 }

```